INSTITUT AFRICAIN— IAI-Togo

Outils de Programmation Web — Laravel 12

Projets de Fin de Module

Sujets, consignes et critères d'évaluation

13 sujets au choix • Groupes de 4 • 1 semaine

Livrable + Soutenance individuelle obligatoire

Dabilibe DOUTI — GLSI-3 — Année académique 2025–2026

Consignes générales applicables à tous les projets

Composition des groupes

Chaque groupe est composé de 4 étudiants. La composition est fixée dès le début et ne peut être modifiée. Chaque groupe choisit librement son sujet parmi les 13 proposés, selon le principe du premier arrivé, premier servi.

GitHub : règles de traitement

- Le dépôt doit être créé dès le premier jour et partagé avec l'enseignant.
- Chaque membre doit avoir des commits nominatifs répartis sur les 2 semaines.
- Un README clair décrit l'installation (.env.example, migration, seed).
- Un historique git entièrement créé en 24 h avant la deadline = zéro pour la partie GitHub.

Soutenance individuelle (10 min / étudiant)

Chaque étudiant est interrogé individuellement sur la partie du code qu'il a développée. Les questions portent sur trois axes :

- Technique : Ex: « Pourquoi tu as utilisé belongsToMany ici ? », « Comment fonctionne ce middleware ? »
- Choix de conception : « Pourquoi tu as séparé cette logique dans un FormRequest ? »
- UX : « Qu'est-ce qui se passe si l'utilisateur soumet ce formulaire deux fois ? »

Attention

- Ne pas pouvoir expliquer une portion de code que l'on a prétendu écrire = zéro sur cette partie.
- L'évaluation est individuelle même si le livrable est rendu en groupe.

Contraintes techniques communes

- Laravel 12 exclusivement (pas de framework CSS imposé — Tailwind, Bootstrap ou CSS maison acceptés)
- Authentification obligatoire via Laravel Breeze ou Jetstream
- Base de données : MySQL ou SQLite, migrations obligatoires
- Les routes doivent respecter les conventions RESTful
- Validation côté serveur obligatoire (Form Requests recommandés)
- Aucune donnée sensible en dur dans le code (utiliser .env)

Projet 1 TontinePro

Application de gestion de tontine et cotisations de groupe

Contexte

- La tontine est une pratique de épargne collective très répandue. Un groupe de personnes cotise régulièrement une somme fixe. À chaque cycle, l'une d'elles perçoit la totalité de la cagnotte selon un ordre préétabli. Vous devez concevoir l'application web qui gère ce processus de bout en bout.

Fonctionnalités attendues

Deux rôles : Organisateur (admin) et Membre.

- Organisateur : créer une tontine (nom, montant/cycle, fréquence, liste des membres), gérer l'ordre des bénéficiaires, enregistrer les paiements de chaque membre, clôturer un cycle.
- Membre : s'inscrire/se connecter, voir son tableau de bord (contributions versées, solde, prochain versement, position dans l'ordre de bénéfice), consulter l'historique du groupe.
- L'application doit gérer plusieurs tontines simultanément (un membre peut appartenir à plusieurs).

Modèle de données attendu (minimum)

Table	Colonnes clés
users	name, email, password, role
tontines	name, amount_per_cycle, frequency, status (active/clôturée)
tontine_user	tontine_id, user_id, beneficiary_order, joined_at
cycles	tontine_id, cycle_number, beneficiary_user_id, opened_at, closed_at
payments	cycle_id, user_id, amount, paid_at, status (payé/en retard)

Contraintes techniques spécifiques

- Utiliser Eloquent belongsToMany avec pivot pour la relation Tontine ↔ Users
- Policé Gate ou Policy : seul l'organisateur peut enregistrer les paiements
- La validation doit empêcher d'enregistrer deux paiements pour le même membre sur le même cycle
- Calcul automatique du statut « en retard » basé sur la date du cycle

Questions UX que vous devrez justifier en soutenance

- Comment indiquez-vous visuellement qu'un membre est en retard de paiement, sans le stigmatiser ouvertement ?
- Le tableau de bord d'un organisateur et celui d'un membre sont-ils identiques ? Pourquoi ?
- Comment confirmez-vous à l'organisateur qu'un paiement a bien été enregistré ?
- Un membre peut-il voir les contributions des autres ? Quel est votre choix et pourquoi ?

Projet 2 TicketFile

Système de prise de ticket et gestion de file d'attente en ligne

Contexte

- Dans de nombreux services (cliniques, bureaux administratifs, banques), les usagers perdent du temps en file d'attente physique. Vous concevez un système web qui permet de prendre un ticket à distance, de suivre sa position dans la file, et aux agents de gérer les guichets depuis leur poste.

Fonctionnalités attendues

Trois rôles : Usager, Agent de guichet, Administrateur.

- Usager : s'inscrire ou accéder sans compte, choisir un service (ex : consultation, paiement, renseignement), obtenir un numéro de ticket, suivre sa position dans la file sur une page qui se rafraîchit automatiquement.
- Agent : voir la file d'attente de son guichet, appeler le ticket suivant, marquer un ticket comme traité ou absent, passer au suivant.
- Admin : créer/modifier les services et guichets, assigner des agents, consulter les statistiques du jour (temps moyen d'attente, tickets traités, tickets annulés).

Modèle de données attendu (minimum)

Table	Colonnes clés
users	name, email, password, role (usager/agent/admin)
services	name, description, is_active
counters	name, service_id, agent_user_id
tickets	ticket_number, service_id, counter_id, user_id (nullable), status (en attente/appelé/traité/absent/annulé), called_at, treated_at, created_at

Contraintes techniques spécifiques

- La page de suivi de l'usager doit se rafraîchir automatiquement via une balise meta refresh ou un appel JavaScript simple (pas de WebSocket requis)
- Le numéro de ticket doit être généré de manière incrémentale et réinitialisé chaque jour
- Utiliser des Policies pour que seul l'agent assigné à un guichet puisse faire avancer sa file
- Les stats admin doivent utiliser des requêtes Eloquent agrégées (avg, count, whereBetween)

Questions UX que vous devrez justifier en soutenance

- Comment rassurez-vous un usager qui attend chez lui sans voir physiquement la salle ?
- Que se passe-t-il visuellement quand c'est le tour de l'usager ?
- Comment l'agent gère-t-il un usager absent ? L'interface le guide-t-elle à prendre une décision rapide ?
- La page de l'agent doit permettre une gestion de 50 tickets en une journée : est-elle efficace ?

Projet 3 **JobLink**

Plateforme de recrutement pour PME locales

Contexte

▸ Les PME locales peinent à diffuser leurs offres d'emploi et à centraliser les candidatures. Les chercheurs d'emploi n'ont pas de plateforme adaptée à leur contexte. Vous concevez un site de mise en relation entre employeurs et candidats, avec un espace personnalisé pour chaque rôle.

Fonctionnalités attendues

Trois rôles : Candidat, Employeur, Modérateur.

- Employeur : créer un profil entreprise, publier des offres d'emploi (titre, description, lieu, type de contrat, date limite), gérer les candidatures reçues (changer les statuts), télécharger les CV.
- Candidat : créer un profil (photo, résumé, compétences), parcourir et filtrer les offres, postuler en joignant un CV (PDF), suivre les statuts de ses candidatures.
- Modérateur : valider ou rejeter les offres publiées avant leur mise en ligne, signaler les profils suspects.

Modèle de données attendu (minimum)

Table	Colonnes clés
users	name, email, password, role, avatar, bio
companies	user_id, name, sector, city, description, logo
job_offers	company_id, title, description, contract_type, city, deadline, status (en attente/publiée/fermée)
applications	job_offer_id, user_id, cv_path, cover_letter, status (reçue/en étude/retenue/rejetée), applied_at

Contraintes techniques spécifiques

- Upload de CV et logo : utiliser Laravel Storage (disk public), valider type MIME et taille (CV : PDF uniquement, max 2 Mo)
- Filtrage des offres : par ville, type de contrat, secteur — utiliser les query scopes Eloquent
- Les statuts de candidature doivent suivre une progression logique et ne pas pouvoir reculer (ex : « retenue » ne revient pas à « reçue »)
- Notification par e-mail (Mailable) quand le statut d'une candidature change

Questions UX que vous devrez justifier en soutenance

▸ Comment communiquez-vous un rejet à un candidat sans le décourager ?

▸ L'employeur reçoit 40 candidatures : comment l'interface l'aide à trier efficacement ?

▸ Sur la page d'une offre d'emploi, dans quel ordre présentez-vous les informations ? Pourquoi ?

▸ Que voit un candidat sur sa page d'accueil après connexion ?

Projet 4 CampusPulse

Suivi des présences et de la progression des étudiants

Contexte

- Un établissement d'enseignement souhaite remplacer les feuilles de présence papier et les relevs de notes imprimés par une application web. Les enseignants saisissent les présences et les notes, les étudiants consultent leur tableau de bord personnel en temps réel.

Fonctionnalités attendues

Trois rôles : Enseignant, Étudiant, Administrateur.

- Enseignant : gérer ses cours, saisir les présences pour une séance (interface rapide pour 30 étudiants), saisir les notes par évaluation, consulter les statistiques par cours.
- Étudiant : tableau de bord personnel (taux de présence par matière, moyennes, évolution sur le semestre, alertes si seuil d'absence dépassé).
- Admin : gérer les utilisateurs, les cours, les filières, consulter les rapports globaux, définir le seuil d'alerte d'absence.

Modèle de données attendu (minimum)

Table	Colonnes clés
users	name, email, password, role, student_number (nullable)
courses	name, code, teacher_user_id, semester, total_hours
course_user	course_id, user_id (pivot étudiants inscrits)
sessions	course_id, date, topic (intitulé de la séance)
attendances	session_id, user_id, status (présent/absent/retard), note
evaluations	course_id, title, coefficient, date
grades	evaluation_id, user_id, score

Contraintes techniques spécifiques

- La saisie de présence doit permettre de traiter 30 étudiants en moins de 2 min (réfléchissez à l'UI)
- Le taux de présence est calculé dynamiquement via Eloquent (pas stocké en base)
- Déclencher une alerte visible dans le tableau de bord étudiant si le taux d'absence dépasse le seuil configuré
- Utiliser des Accessors Eloquent pour formater les moyennes (ex : arrondi à 2 décimales)

Questions UX que vous devrez justifier en soutenance

- Comment saisir 30 présences en moins de 2 minutes ? Décrivez votre solution d'interface.
- Comment visualisez-vous un taux d'absence alarmant sans paniquer l'étudiant ?
- Sur le tableau de bord étudiant, quelle est la première chose qu'il doit voir à l'ouverture ?
- Comment différenciez-vous visuellement un « absent » d'un « retard » ?

Projet 5 AskCampus

Plateforme d'entraide académique entre étudiants

Contexte

▸ Les étudiants manquent d'un espace structuré pour poser leurs questions de cours et partager des réponses entre pairs. Vous concevez une plateforme de type forum structuré, où les questions sont classées par tags, les réponses votées, et les meilleures mises en avant.

Fonctionnalités attendues

Deux rôles : Étudiant, Modérateur.

- Étudiant : poser une question (titre, description, tags), répondre aux questions des autres, voter pour les questions et les réponses, marquer la meilleure réponse à sa propre question, consulter son profil (réputation, questions posées, réponses données).
- Modérateur : supprimer du contenu inapproprié, gérer les tags, fermer/rouvrir des questions.
- Moteur de recherche interne : recherche par mots-clés dans le titre et le corps, filtre par tag.

Modèle de données attendu (minimum)

Table	Colonnes clés
users	name, email, password, role, reputation (score calculé)
questions	user_id, title, body, is_solved, views, created_at
answers	question_id, user_id, body, is_accepted, created_at
votes	user_id, votable_id, votable_type, value (+1 ou -1) — relation polymorphe
tags	name, slug, description
question_tag	question_id, tag_id (pivot)

Contraintes techniques spécifiques

- Implémenter la relation polymorphe (MorphTo) pour les votes : un vote peut s'appliquer à une question ou une réponse
- La recherche doit utiliser les clauses where/orWhere sur title et body (pas de moteur de recherche externe requis)
- Ordonner les réponses par score de votes décroissant, la réponse acceptée toujours en premier
- Un utilisateur ne peut voter qu'une fois par question/réponse (contrainte en base + validation Laravel)

Questions UX que vous devrez justifier en soutenance

▸ Comment guidez-vous un étudiant pour qu'il pose une bonne question (pas juste un champ vide) ?

▸ Comment l'interface suggère-t-elle qu'une question similaire existe déjà, avant la soumission ?

▸ Sur une page de question avec 8 réponses, comment la meilleure ressort-elle visuellement ?

▸ Que représente la réputation d'un utilisateur et comment l'affichez-vous sans que ça devienne une compétition toxique ?

Projet 6 RoomBook

Réservation de salles et de matériels pour une institution

Contexte

- Un établissement dispose de plusieurs salles de cours et d'un parc de matériels (vidéoprojecteurs, tableaux interactifs, micros). Les enseignants réservent ces ressources en ligne. Un responsable valide les demandes. Les conflits de réservation doivent être détectés automatiquement.

Fonctionnalités attendues

Trois rôles : Enseignant, Responsable, Administrateur.

- Enseignant : voir le planning des disponibilités (vue semaine), faire une demande de réservation (salle + matériel facultatif, créneau, motif), suivre le statut de ses demandes, annuler une réservation acceptée.
- Responsable : voir les demandes en attente, accepter ou refuser avec un motif, voir le planning global.
- Admin : gérer les salles (capacité, équipements, disponibilité), gérer le matériel, gérer les utilisateurs.

Modèle de données attendu (minimum)

Table	Colonnes clés
users	name, email, password, role
rooms	name, capacity, building, is_available
equipment	name, quantity, is_available
bookings	user_id, room_id, starts_at, ends_at, purpose, status (en attente/acceptée/refusée/annulée), rejection_reason
booking_equipment	booking_id, equipment_id, quantity (pivot)

Contraintes techniques spécifiques

- Détection de conflits : à la soumission, vérifier qu'aucune réservation acceptée ne chevauche le créneau demandé pour la même salle
- Validation personnalisée (Rule custom) : l'heure de fin doit être strictement supérieure à l'heure de début
- La vue planning doit afficher les réservations de la semaine courante, naviguable semaine par semaine
- Notification email (Mailable) à l'enseignant lors de l'acceptation ou du refus

Questions UX que vous devrez justifier en soutenance

- Comment montre-t-on en un coup d'œil les disponibilités d'une salle sur une semaine ?
- À quel moment dans le parcours de réservation informez-vous l'utilisateur d'un conflit ?
- Comment différenciez-vous visuellement une salle « réservée » d'une salle « en attente de validation » ?
- Que se passe-t-il si l'enseignant tente d'annuler une réservation qui commence dans 30 minutes ?

Projet 7 EventPass

Gestion d'événements et billetterie en ligne

Contexte

- Une association ou une institution organise régulièrement des événements (conférences, concerts, ateliers). Les inscriptions se font encore par téléphone ou sur papier. Vous créez l'application web qui gère l'affichage des événements, les inscriptions en ligne et la délivrance d'un pass numérique unique par participant.

Fonctionnalités attendues

Trois rôles : Organisateur, Participant, Administrateur.

- Organisateur : créer un événement (titre, description, date, lieu, image, nombre de places, tarif ou gratuit), voir la liste des inscrits, invalider ou réactiver un pass.
- Participant : parcourir les événements à venir, s'inscrire, récupérer son pass (code alphanumérique unique à présenter à l'entrée), consulter ses inscriptions passées.
- Admin : modérer les événements publiés, voir les statistiques globales (taux de remplissage, événements les plus populaires).

Modèle de données attendu (minimum)

Table	Colonnes clés
users	name, email, password, role
events	organizer_id, title, description, starts_at, location, capacity, price, image_path, status (brouillon/publié/annulé/terminé)
registrations	event_id, user_id, pass_code (unique), status (validé/annulé), registered_at

Contraintes techniques spécifiques

- Générer le pass_code avec Str::upper(Str::random(8)) et garantir l'unicité en base
- Empêcher l'inscription si capacity atteinte (verroué au niveau validation, pas seulement en front)
- Scope Eloquent : upcoming() (starts_at > now()), past(), full() (registrations count >= capacity)
- Un participant ne peut s'inscrire qu'une seule fois au même événement (contrainte unique en base)

Questions UX que vous devrez justifier en soutenance

- Comment affichez-vous le nombre de places restantes — et que se passe-t-il visuellement quand il n'en reste que 2 ?
- Où et comment le participant retrouve-t-il son pass facilement après inscription ?
- Que se passe-t-il si l'événement affiche « complet » alors que l'utilisateur était en train de s'inscrire ?
- Comment différenciez-vous visuellement un événement passé d'un événement à venir sur la page d'accueil ?

Projet 8 StockPilot

Gestion d'inventaire et de stock pour PME

Contexte

▸ Une petite boutique ou pharmacie gère son stock manuellement dans des cahiers. Les erreurs sont fréquentes et les ruptures détectées trop tard. Vous créez l'outil web qui centralise le catalogue produits, les mouvements de stock et les alertes de seuil critique.

Fonctionnalités attendues

Deux rôles : Gérant, Employé.

- Gérant : tableau de bord (produits en alerte, valeur totale du stock, mouvements récents), gérer les catégories et les produits, définir le seuil d'alerte par produit, consulter les rapports (top produits sortis, historique par période).
- Employé : enregistrer un mouvement de stock (entrée fournisseur, sortie vente, ajustement inventaire) avec motif et quantité, rechercher un produit, consulter le stock actuel.

Modèle de données attendu (minimum)

Table	Colonnes clés
categories	name, description
products	category_id, name, unit, current_stock, alert_threshold, unit_price
stock_movements	product_id, user_id, type (entrée/sortie/ajustement), quantity, reason, created_at

Contraintes techniques spécifiques

- current_stock doit être mis à jour à chaque mouvement via un Observer Eloquent (ProductObserver)
- Scope : Product::belowThreshold() pour les alertes du tableau de bord
- Accessor : stock_status retourne « critique » / « faible » / « normal » selon le seuil
- Valider qu'une sortie ne rend pas le stock négatif (règle de validation personnalisée)

Questions UX que vous devrez justifier en soutenance

▸ Comment le gérant repère-t-il en un coup d'œil les produits critiques sur son tableau de bord ?

▸ Comment l'employé saisit-il un mouvement sur 20 articles différents rapidement ?

▸ Que se passe-t-il si l'employé tente une sortie qui mettrait le stock à zéro — comment l'interface le gère-t-elle ?

▸ Que voit un employé en premier à la connexion, et pourquoi est-ce différent du gérant ?

Projet 9 PetitChef

Commande de repas maison — traiteur local en ligne

Contexte

- Des traiteurs locaux et cuisiniers à domicile proposent leurs plats chaque jour, mais les commandes se font par téléphone avec des erreurs fréquentes. Vous créez la plateforme web qui affiche le menu du jour, gère les commandes et permet au cuisinier de suivre sa production en temps réel.

Fonctionnalités attendues

Trois rôles : Cuisinier, Client, Administrateur.

- Cuisinier : publier les plats du jour (nom, description, prix, quantité disponible, photo), voir et gérer les commandes reçues (statuts : reçue / en préparation / prête / livrée), clôturer le service du jour.
- Client : voir le menu du jour, ajouter des plats au panier, passer la commande (avec heure de récupération souhaitée), suivre le statut de sa commande.
- Admin : valider les profils cuisiniers, gérer les signalements, consulter les statistiques.

Modèle de données attendu (minimum)

Table	Colonnes clés
users	name, email, password, role, phone
dishes	cook_id, name, description, price, available_qty, photo_path, is_active, served_date
orders	client_id, cook_id, total_price, pickup_time, status (reçue/en préparation/prête/livrée/annulée), note_client
order_dish	order_id, dish_id, quantity, unit_price (pivot snapshot du prix)

Contraintes techniques spécifiques

- Décrémenter available_qty lors de la commande et refuser si stock insuffisant (transaction DB pour éviter la survente)
- Stocker unit_price dans le pivot — le prix en base peut changer, la commande garde le prix de l'achat
- Scope : Dish::ofTheDay() filtré sur served_date = today() et is_active = true
- La clôture du service désactive tous les plats du jour restants (action de masse Eloquent)

Questions UX que vous devrez justifier en soutenance

- Comment indiquez-vous qu'un plat est « bientôt épuisé » vs « épuisé » sur la page menu ?
- Comment le cuisinier gère-t-il 15 commandes simultanées sans se perdre ?
- Que se passe-t-il côté client si la quantité d'un plat devient zéro pendant qu'il est en train de commander ?
- La page du cuisinier et celle du client partagent-elles le même layout Blade ? Justifiez.

Projet 10 LibraFlow

Gestion de bibliothèque scolaire numérique

Contexte

▸ Une école gère encore sa bibliothèque sur des registres papier. Les étudiants ne savent pas si un livre est disponible avant de se déplacer. Vous créez le système web qui numérise le catalogue, les emprunts, les retours et les réservations.

Fonctionnalités attendues

Deux rôles : Bibliothécaire, Lecteur (+ Admin).

- Bibliothécaire : gérer le catalogue (livres et nombre d'exemplaires), enregistrer les emprunts et les retours, voir les retards, gérer les réservations en file d'attente.
- Lecteur : rechercher un livre (par titre, auteur, catégorie), voir la disponibilité en temps réel, réserver un livre si tous les exemplaires sont empruntés, consulter son historique et ses éventuelles pénalités de retard.
- Admin : paramétrer la durée d'emprunt (ex. 14 jours) et le montant des pénalités journalières.

Modèle de données attendu (minimum)

Table	Colonnes clés
books	title, author, isbn, category, total_copies, available_copies, cover_image
loans	book_id, user_id, borrowed_at, due_at, returned_at (nullable), penalty_amount
reservations	book_id, user_id, reserved_at, status (en attente/notifié/annulée)
settings	key, value (pour durée_emprunt, pénalité_journalière, etc.)

Contraintes techniques spécifiques

- available_copies est décrémenté à l'emprunt et incrémenté au retour (Observer Eloquent)
- Scope : Loan::overdue() (due_at < now() et returned_at is null), calcul automatique de penalty_amount
- Lors d'un retour, vérifier s'il existe une réservation en attente et notifier le premier en file (Mailable)
- Utiliser Carbon pour toutes les manipulations de dates d'échéance

Questions UX que vous devrez justifier en soutenance

▸ Si un livre a 3 exemplaires dont 1 disponible, comment l'affichez-vous clairement sur la fiche livre ?

▸ Comment communiquez-vous une pénalité de retard sans être agressif ni culpabilisant ?

▸ La page de recherche : quels filtres proposez-vous, dans quel ordre, et pourquoi ?

▸ Un lecteur réserve un livre déjà réservé par quelqu'un d'autre — que lui dites-vous et que faites-vous ?

Projet 11 **MentorLink**

Plateforme de mentorat académique entre étudiants

Contexte

▸ Des étudiants seniors souhaitent partager leur expérience avec des juniors (révisions, orientation, projets). Il n'existe pas de cadre structuré pour ça. Vous créez la plateforme qui met en relation mentors et mentorés, gère la réservation de créneaux et recueille les évaluations post-session.

Fonctionnalités attendues

Trois rôles : Mentor, Mentoré, Administrateur.

- Mentor : créer son profil (domaines de compétences, bio, tarif ou gratuit), définir ses disponibilités (jours + plages horaires), confirmer ou refuser une demande, marquer une session comme terminée.
- Mentoré : parcourir les profils (filtrer par domaine), consulter les créneaux disponibles d'un mentor, réserver une session, évaluer la session après (note 1–5 + commentaire), annuler une réservation en avance.
- Admin : valider les profils mentors avant publication, gérer les signalements, voir les statistiques (mentors les mieux notés, sessions par domaine).

Modèle de données attendu (minimum)

Table	Colonnes clés
users	name, email, password, role, bio, avatar
mentor_profiles	user_id, domains (JSON ou table pivot), hourly_rate, is_validated, average_rating (calculé)
availabilities	mentor_id, day_of_week (0-6), start_time, end_time
sessions	mentor_id, mentee_id, scheduled_at, duration_min, status (en attente/confirmée/terminée/annulée)
reviews	session_id, reviewer_id, rating (1-5), comment, created_at

Contraintes techniques spécifiques

- Détecter les conflits de réservation : une session ne peut pas être réservée si le mentor a déjà une session confirmée sur ce créneau
- average_rating est recalculé via un accessor Eloquent (average des reviews du mentor)
- Policy : seul le mentoré concerné peut déposer un avis sur sa propre session
- Afficher les disponibilités d'un mentor en excluant les créneaux déjà réservés

Questions UX que vous devrez justifier en soutenance

▸ Comment un junior choisit-il le bon mentor parmi 20 profils affichés ?

▸ Comment affichez-vous les créneaux disponibles d'un mentor de manière intuitive (pas juste une liste de texte) ?

▸ Comment affichez-vous la note d'un mentor qui n'a encore aucun avis — sans le désavantager ?

▸ Que se passe-t-il si un mentor refuse une session déjà confirmée — comment en informez-vous le mentoré ?

Projet 12 CampusShop

Petites annonces et troc entre étudiants

Contexte

▸ Les étudiants revendent des cours, du matériel scolaire ou des vêtements entre eux via des groupes WhatsApp désorganisés. Vous créez une plateforme fermée (réservée aux membres de l'établissement) pour poster des annonces, contacter les vendeurs et gérer les transactions informelles.

Fonctionnalités attendues

Deux rôles : Membre, Modérateur.

- Membre : créer une annonce (type : vente / don / échange, titre, description, prix, jusqu'à 3 photos), contacter le vendeur via messagerie interne, marquer son annonce (disponible / réservé / vendu), mettre en favoris, signaler une annonce.
- Modérateur : valider les annonces avant publication, traiter les signalements (masquer, avertir, supprimer), gérer les membres exclus temporairement.

Modèle de données attendu (minimum)

Table	Colonnes clés
users	name, email, password, role, student_id, is_active
listings	user_id, title, description, type (vente/don/échange), price, status (publiée/réservée/vendue/archivée), views_count
listing_photos	listing_id, path, order (1-3)
conversations	listing_id, buyer_id, seller_id, last_message_at
messages	conversation_id, sender_id, body, read_at, created_at
favorites	user_id, listing_id (pivot)

Contraintes techniques spécifiques

- Upload jusqu'à 3 photos par annonce (validation MIME, taille max 1 Mo chacune), une par une ou en groupe
- Messagerie : une conversation est liée à une annonce et deux utilisateurs (unique sur ces 3 clés)
- Marquer les messages comme lus (read_at) quand le destinataire ouvre la conversation
- Scope : Listing::active() (statut publiée), Listing::mine() (de l'utilisateur connecté)

Questions UX que vous devrez justifier en soutenance

▸ Comment différenciez-vous visuellement une annonce « réservée » d'une annonce « disponible » dans la liste ?

▸ Un vendeur reçoit 5 messages sur la même annonce : comment l'interface l'aide à tout gérer ?

▸ La page d'accueil : quelles annonces affichez-vous en premier et selon quel critère ?

▸ Comment évitez-vous qu'un étudiant poste le même article deux fois sans interdire explicitement ?

Projet 13 **PlanifPro**

Gestionnaire de tâches et de projets d'équipe

Contexte

▸ Une petite équipe ou association gère ses projets par e-mail et groupes WhatsApp : personne ne sait qui fait quoi, les délais sont ratés. Vous créez un outil web de gestion de tâches collaboratif, avec une vue par colonnes de statut (kanban simplifié) et un espace de commentaires par tâche.

Fonctionnalités attendues

Trois rôles : Chef de projet, Membre, Administrateur.

- Chef de projet : créer des projets, inviter des membres, créer des tâches (titre, description, priorité, membre assigné, deadline), changer le statut des tâches (À faire / En cours / En révision / Terminé), voir le tableau de bord du projet (avancé, en retard).
- Membre : voir ses tâches assignées (dans tous ses projets), mettre à jour le statut d'une tâche, ajouter des commentaires sur une tâche.
- Admin : gérer les utilisateurs, archiver des projets inactifs, voir les statistiques globales.

Modèle de données attendu (minimum)

Table	Colonnes clés
projects	owner_id, name, description, status (actif/archivé), deadline
project_user	project_id, user_id, role (chef/membre) — pivot
tasks	project_id, assigned_to, title, description, priority (basse/normale/haute/urgente), status (à faire/en cours/en révision/terminé), deadline, position
comments	task_id, user_id, body, created_at

Contraintes techniques spécifiques

- Policy : seul le chef de projet ou l'assigné peut modifier le statut d'une tâche
- Scope : Task::overdue() (deadline passée et statut != terminé), Task::mine() (assigné à l'utilisateur connecté)
- Le champ position permet de trier les tâches dans chaque colonne (ordonner manuellement)
- Le changement de statut ne peut pas « reculer » d'une étape (Terminé ne revient pas à À faire sans droits de chef)

Questions UX que vous devrez justifier en soutenance

▸ Vue kanban : comment distingue-t-on visuellement une tâche urgente d'une tâche normale dans la même colonne ?

▸ Un membre a 12 tâches sur 3 projets différents : où voit-il ce qui est urgent sans naviguer entre les projets ?

▸ Le déplacement d'une tâche entre colonnes : comment l'implémentez-vous sans drag-and-drop complexe ?

▸ Que se passe-t-il si un chef de projet archive un projet qui a encore des tâches en cours assignées à des membres ?

Grille d'évaluation — 20 points

La note finale de chaque étudiant est composée d'une note de groupe (livrable) et d'une note individuelle (soutenance). Une note de soutenance inférieure à 4/10 entraîne un plafonnement de la note finale à 10/20, quelle que soit la qualité du livrable.

Répartition des points

Critère	Description	Points
NOTE DE GROUPE (livrable)		12 pts
Réalisation fonctionnelle	Toutes les fonctionnalités demandées sont implémentées et fonctionnelles	5
Qualité technique	Code Laravel propre : routes RESTful, Form Requests, Policies, séparation des responsabilités	3
Qualité UX / Interface	Navigation claire, feedback utilisateur, gestion des erreurs, cohérence visuelle	2
GitHub & processus	Historique de commits réparti, README complet, wireframes livrés en S1	2
NOTE INDIVIDUELLE (soutenance)		8 pts
Maîtrise du code	L'étudiant explique clairement la portion de code qu'il a développée	3
Choix techniques	Justification des décisions Laravel (relations, validation, middleware…)	3
Réponse aux questions UX	L'étudiant justifie les choix d'interface de sa partie	2

Grille de progression — soutenance individuelle

Utilisez cette grille pour noter chaque étudiant pendant la soutenance.

Dimension	Indicateurs observés	Note /8
Maîtrise du code (3 pts)	Peut relire son code à voix haute et l'expliquer ligne à ligne. Répond aux questions « pourquoi ce choix ? » sans hésitation. Sait localiser une fonctionnalité dans le projet.	/ 3
Choix techniques (3 pts)	Justifie l'utilisation des features Laravel (relation Eloquent, middleware, FormRequest, Policy). Comprend les alternatives qu'il aurait pu faire. Identifie les limites de son implémentation.	/ 3
Réponse UX (2 pts)	Justifie les choix de présentation et d'interaction dans sa partie. Peut proposer une amélioration si l'enseignant soulève un problème.	/ 2

Signaux d'alerte pendant la soutenance

- L'étudiant ne sait pas où se trouve le code d'une fonctionnalité qu'il prétend avoir écrite.

- L’étudiant ne peut pas expliquer pourquoi une relation est définie dans tel sens (hasMany vs belongsTo).
- L’étudiant découvre le code en direct (scrolle longtemps avant de trouver).
- À la question « Que se passe-t-il si l’utilisateur fait X ? », l’étudiant ne sait pas répondre sans ouvrir l’application.